

Running The Mainframe

Everything a facilitator needs in the room: what each task asks for, a scaffolded way to help someone stuck without just giving them the answer, the full answer key for when that's genuinely the right call, what the chat systems can and can't actually do, and troubleshooting for "it's not working" moments that are user error, not a broken build.

TIMING

HELLO, COMPUTER

CONSTRUCT

VAULT

LEDGER

FRONT

HELPDESK

COPILOT MODULES

INTRO SLIDES

CERTIFICATES & MY TROPHIES

DASHBOARD STRUCTURE

HELP & HINTS — WHAT WORKS

HELPDESK CHAT — WHAT WORKS

TROUBLESHOOTING

ALL FILES

THE DASHBOARD'S "I'M INTERESTED IN..." PICKER

A small feature on `index.html` — five toggleable chips (Coding, Web Design, AI Prompting, Data & Dashboards, Debugging) that highlight matching module cards and dim the rest when clicked, multi-select. Several modules genuinely belong to more than one category: Ledger is tagged both Coding and Data (it's the array/query module, and directly connects to the Excel dashboard exercise); Helpdesk is tagged both Coding and Debugging (it's the one module that's purely about fixing rather than writing); the Excel/Outlook exercise is tagged both AI Prompting and Data. This is genuinely useful mainly once there are enough modules that browsing all of them at once feels like a lot — point newcomers at it if they seem unsure where to start.

THE 25-MINUTE SLOT

Five stations, roughly 5 minutes each, with a short buffer for moving between them. The Construct taster has a visible on-screen countdown set to 6 minutes — the other four don't enforce a timer, so you'll need to call time yourself.

Construct ~5m

Vault ~5m

Ledger ~5m

Front ~5m

Helpdesk ~5m

Nobody needs to finish every task in every activity — the point is a real taste of each, not full completion. If someone's clearly engaged and close to finishing when time's up, let them carry on; don't cut mid-flow over a minute or two. Construct and Vault now have 5 tasks each rather than 3 — genuinely finishing either one in 5 minutes is a stretch goal, not an expectation.

HELLO, COMPUTER

What it is: `hello-world.html`, first on the dashboard, mandatory before everything else. A 3-minute warm-up that explains what a "console" actually is — plain-English, no assumed knowledge — then walks through a real `console.log("Hello, World!")` exercise, then a second step swapping in your own name. Ends with a short recap plus a "why this matters for real jobs" note.

This exists specifically because "people don't know what a console is" was repeated, direct feedback.

Every other coding activity assumes this has already been done — several of them (Construct's terminal, in particular) explicitly call back to it ("This works like the console from Hello, Computer..."). If someone skips straight to another

activity without doing this first, that callback language may not land, so it's worth actually enforcing "start here" rather than treating it as optional, even though nothing technically blocks skipping it.

Validation is deliberately forgiving. Smart quotes from phone/tablet autocorrect are normalised so they don't unfairly fail someone typing on a touchscreen. Empty submissions get a specific "the box is empty" nudge rather than a generic error. Typos get a constructive retry message, not just "wrong."

THE CONSTRUCT

What it is: navigating a hidden rover across a grid by writing real JavaScript — `moveForward()`, `turnLeft()`, `turnRight()` — building from a single command up to a loop, and explicitly showing that a loop and manual repeated calls are equally valid solutions. Covers functions, sequencing, parameters, and loops.

The goal genuinely wasn't obvious as a goal — feedback confirmed this, and checking the actual rendering confirmed why. The target cell was only a soft colour difference plus an abstract pulsing ring, with no icon and no plain-language statement of the objective anywhere on screen — brief text called it "the exit node," a term that never visually connected to that pulsing circle. Fixed in both the taster and full build: the target cell now shows an actual  flag icon, a plain " Your goal: get the rover to the flag" banner sits directly above the grid, and every brief, hint, and terminal log message was updated to consistently say "flag" instead of "exit"/"exit node," so the words and the visual reinforce each other. Confirmed the flag renders on the correct cell across every level in both files, including the newest one (The Rectangle), and confirmed the win condition and log messages still work correctly end to end.

#	TASK	ASKS FOR
1	Wake Up	call <code>moveForward()</code> once
2	The Path	four <code>moveForward()</code> calls in a row
3	The Turn	move, turn, move again — combining direction changes
4	The Relay	two turns — three straight stretches chained together
5	The Repeat	a diagonal staircase — solvable manually or with a loop, deliberately
— take-home extension diverges from level 4 onward, see below —		
Bonus	The Glitch	fix a loop that runs one time too few (debugging, not new syntax)

Important — the take-home is not "the same plus one task": levels 1–3 (Wake Up, The Path, The Turn) are byte-identical between the taster and the take-home. Levels 4 and 5 genuinely diverge — the taster has The Relay and The Repeat; the take-home has Agents and The Loop instead. A participant doing both will redo 60% of the content before hitting anything new, which is exactly why it can feel like barely an extension even though two levels do differ, not one. A brand new 6th level, The Rectangle, was added specifically to address this — see the take-home table below.

The Repeat is worth knowing about specifically: the path staircases diagonally rather than running in a straight line — deliberately, after feedback that a straight run didn't feel meaningfully harder than earlier tasks, just longer. The repeating unit is four commands (move, turn, move, turn back), not one, so the manual solution is genuinely long (16 lines) while the loop version stays at 4 — a much more dramatic and honest demonstration of why loops

matter. The checking logic still only ever looks at where the rover ends up, never at what code got it there, so both the 16-line manual solution and the 4-line loop genuinely pass.

SCAFFOLDED HELP

TIER 1 — ASK, DON'T TELL

"How many cells away is the exit? What do you think that number needs to be in your code?"

TIER 2 — POINT AT THE TOOL

Point them at the in-app Hint button before adding anything yourself — it escalates through three tiers on its own, ending in the full answer if they click it enough times.

TIER 3 — NARROW IT DOWN

"You've got the right idea, but count your `moveForward()` calls again — you're one short." Still make them fix it themselves.

ANSWER KEY

Wake Up

```
moveForward();
```

The Path

```
moveForward();  
moveForward();  
moveForward();  
moveForward();
```

The Turn

```
moveForward();  
moveForward();  
moveForward();  
turnRight();  
moveForward();  
moveForward();  
moveForward();
```

The Relay

```
moveForward();  
moveForward();  
turnRight();  
moveForward();  
moveForward();  
turnLeft();  
moveForward();  
moveForward();
```

The Repeat (either solution passes)

```
// Loop (4 lines):
for (let i = 0; i < 4; i++) {
  moveForward();
  turnRight();
  moveForward();
  turnLeft();
}

// Manual: the same 4-line block repeated 4 times (16 lines) — also passes.
```

Bonus — The Glitch

Change the loop condition from `i < 3` to `i < 4` — the exit is 4 cells away, the loop only ran 3 times.

TAKE-HOME EXTENSION — LEVELS 4 ONWARD DIFFER ENTIRELY

#	TASK	ASKS FOR
1-3	Wake Up, The Path, The Turn	identical to the taster — not new content
4	Agents	plan a route around obstacles blocking the direct path — logic/planning, not a new command
5	The Loop	a staircase requiring a loop body with more than one command inside it
6	The Rectangle	new — an L-shaped route: one straight stretch, a turn, then a second straight stretch of a different length. Requires two separate loops run back to back, each with its own count — the natural next step after a single repeating loop.

Agents

```
turnRight();
moveForward();
moveForward();
moveForward();
moveForward();
turnLeft();
moveForward();
moveForward();
moveForward();
moveForward();
```

Several valid routes exist — the walls only need to be routed around, not solved one specific way. This is one that works.

The Loop

```
for (let i = 0; i < 5; i++) {
  moveForward();
  turnRight();
  moveForward();
```

```
turnLeft();  
}
```

The Rectangle (new)

Start (0,0) facing East, target (5,3). Walls block turning south anywhere before x=5, so the only route is: straight east to x=5, turn, then straight south to y=3.

```
for (let i = 0; i < 5; i++) {  
  moveForward();  
}  
turnRight();  
for (let i = 0; i < 3; i++) {  
  moveForward();  
}
```

Wall placement was mathematically verified, not eyeballed: walls sit at every point along y=0 for x=0 through x=4 specifically to block turning south early — confirmed by simulating the intended solution end to end before this shipped, and separately confirming every early turn-south point is genuinely blocked while x=5 (the only valid turning point) is genuinely open.

THE VAULT

What it is: writing real functions to patch broken business logic in a bank's risk-checking system, then running real test cases against them. Covers functions, return values, booleans, conditionals, and — new — a two-parameter conditional and a built-in maths function.

#	ON-SCREEN TITLE	FUNCTION	ASKS FOR
1	Add a Service Charge	<code>calculateTotal</code>	add a flat £5 fee to an amount
2	Spot a High-Risk Transaction	<code>isHighRisk</code>	return true/false for amounts over £10,000
3	Sort a Transaction by Size	<code>categoriseTransaction</code>	three-way if/else banding
4	Add On a Late Fee	<code>applyLateFee</code>	two parameters, one used as a conditional gate
5	Round to the Nearest Pound	<code>roundToNearestPound</code>	introduces <code>Math.round()</code>
— take-home extension adds 3 more, see below —			
6	Apply a Loyalty Discount	<code>applyLoyaltyDiscount</code>	multiple parameters, one a boolean flag
7	Calculate Interest Owed	<code>calculateInterest</code>	two-parameter arithmetic
8	Flag Suspicious Activity	<code>flagSuspiciousActivity</code>	compound condition with <code>&&</code>

SCAFFOLDED HELP

TIER 1 — ASK, DON'T TELL

"What does the checklist say your function actually returned? Is that different from what it expected?"

TIER 2 — THE USUAL SUSPECTS

Missing `return` , or `>` vs `>=` not matching the brief exactly — ask them to re-read the brief's wording once more before anything else.

TIER 3 — NARROW IT DOWN

Point at the specific failing test row and the exact value it returned vs expected — that's usually enough for someone to spot it themselves.

ANSWER KEY

calculateTotal

```
function calculateTotal(amount) {  
  return amount + 5;  
}
```

isHighRisk

```
function isHighRisk(amount) {  
  return amount > 10000;  
}
```

categoriseTransaction

```
function categoriseTransaction(amount) {  
  if (amount > 10000) return 'flagged';  
  if (amount >= 1000) return 'review';  
  return 'standard';  
}
```

applyLateFee

```
function applyLateFee(amount, daysLate) {  
  if (daysLate > 0) return amount + 10;  
  return amount;  
}
```

roundToNearestPound

```
function roundToNearestPound(amount) {  
  return Math.round(amount);  
}
```

applyLoyaltyDiscount (extension)

```
function applyLoyaltyDiscount(amount, isPremium) {  
  return isPremium ? amount * 0.9 : amount;  
}
```

calculateInterest (extension)

```
function calculateInterest(balance, rate) {  
  return balance * (rate / 100);  
}
```

flagSuspiciousActivity (extension)

```
function flagSuspiciousActivity(amount, country) {  
  return amount > 5000 && country !== 'UK';  
}
```

THE LEDGER

What it is: writing functions that loop through a real, visible list of transactions to answer questions — totals, filters, counts. Each function also gets tested against a second, hidden dataset, so hardcoding the visible numbers doesn't pass. Covers variables, arrays, objects, and loops.

If this was confusing before, it's had a real fix: the panel showing the data now explicitly spells out that row 1 (`groceries, £120`) is `transactions[0].category` and `transactions[0].amount` in code — that bridge was missing before and was very likely the actual cause of confusion, not the difficulty of the task itself. There was also a genuine layout bug where, on any narrow screen, the code editor rendered *above* the data table — fixed so the data always shows first.

#	ON-SCREEN TITLE	FUNCTION	ASKS FOR
1	Find the First Transaction	<code>firstTransactionAmount</code>	single-item access, no loop yet — the warm-up
2	Add Up Every Transaction	<code>totalSpend</code>	accumulator pattern — sum every amount
3	Total One Category	<code>sumByCategory</code>	filtered sum, one category
4	Count the Big Transactions	<code>countHighValue</code>	filtered count against a threshold
— take-home extension adds 3 more —			
5	Find the Biggest Transaction	<code>largestTransaction</code>	running maximum
6	Work Out the Average	<code>averageTransaction</code>	division + rounding
7	Total Several Categories at Once	<code>sumByCategories</code>	filter against an array of categories

SCAFFOLDED HELP

TIER 1 — ASK, DON'T TELL

"Where's your total variable declared — inside the loop, or before it?" This catches the single most common bug here on its own.

TIER 2 — READ IT ALOUD

Get them to read their `if` condition out loud in plain English — mismatches between what they meant and what they typed usually surface immediately.

TIER 3 — NARROW IT DOWN

If the visible-ledger test passes but the hidden one fails, that's a strong sign they hardcoded a number instead of writing real logic — worth saying so directly.

ANSWER KEY

totalSpend

```
function totalSpend(transactions) {
  let total = 0;
  for (let i = 0; i < transactions.length; i++) {
    total += transactions[i].amount;
  }
  return total;
}
```

sumByCategory

```
function sumByCategory(transactions, category) {
  let total = 0;
  for (let i = 0; i < transactions.length; i++) {
    if (transactions[i].category === category) {
      total += transactions[i].amount;
    }
  }
  return total;
}
```

countHighValue

```
function countHighValue(transactions, threshold) {
  let count = 0;
  for (let i = 0; i < transactions.length; i++) {
    if (transactions[i].amount > threshold) {
      count++;
    }
  }
  return count;
}
```

largestTransaction (extension)

```
function largestTransaction(transactions) {
  let biggest = 0;
  for (let i = 0; i < transactions.length; i++) {
    if (transactions[i].amount > biggest) {
      biggest = transactions[i].amount;
    }
  }
  return biggest;
}
```

averageTransaction (extension)

```
function averageTransaction(transactions) {
  let total = 0;
  for (let i = 0; i < transactions.length; i++) {
    total += transactions[i].amount;
  }
  return Math.round(total / transactions.length);
}
```

sumByCategories (extension)

```
function sumByCategories(transactions, categories) {
  let total = 0;
  for (let i = 0; i < transactions.length; i++) {
    if (categories.includes(transactions[i].category)) {
      total += transactions[i].amount;
    }
  }
  return total;
}
```

THE FRONT

What it is: writing real CSS against a live preview to hit stated visual goals — checked by reading actual computed styles, not exact pixel matching. Covers selectors, the box model, typography, and flexbox.

Two real changes worth knowing about: there's now a colour picker next to the editor — click a swatch and it shows the hex code to copy, and the brief also says a plain colour name like `navy` works exactly as well as a hex code. Separately, the worked examples used to use values that would pass the actual checks if copied verbatim with just the selector renamed — that's fixed. The worked examples still show the right properties, but with deliberately small or slightly wrong values (e.g. `justify-content: center` instead of `space-between`), so a blind copy-paste no longer passes — genuine adaptation is required.

#	SCREEN	ASKS FOR
1	The Panel	background, rounded corners, padding
2	The Numbers	font-size, font-weight
3	The Layout	flexbox row, space-between, centred
Bonus	The Stylesheet Bug	find a real CSS typo (debugging, not new syntax)

Each screen also has a collapsed "✱ Try these extensions" box with two optional, unchecked stretch ideas — a hover effect, a box-shadow, a text-shadow, and so on. Nobody needs to do these to pass; they're there for anyone who finishes early. The take-home extension restructures the taster into **one continuous page** (header, nav, card, transaction list) instead of resetting between screens, with 2 extensions per section (8 total). Finishing all three now leads to a done screen with real options — 🏹 **Try Again** resets everything, ✂ **Bonus Round** starts a genuine CSS debugging challenge (see below) rather than a dead end.

SCAFFOLDED HELP

TIER 1 — ASK, DON'T TELL

"What does the preview actually look like right now versus what you're going for?" Get them looking at the live preview,

not just the code.

TIER 2 — THE USUAL SUSPECTS

A missing semicolon silently breaks everything after it in that block — if nothing they add seems to work, check for one missing semicolon above it.

TIER 3 — NARROW IT DOWN

`display: flex` has to go on the parent (the container), not the item inside it — this is the single most common flexbox mistake in the room.

ANSWER KEY

The Panel

```
.account-card {  
  background: #1a1f29;  
  border-radius: 12px;  
  padding: 24px;  
}
```

The Numbers

```
.account-balance {  
  font-size: 32px;  
  font-weight: 700;  
}
```

The Layout

```
.account-card {  
  display: flex;  
  justify-content: space-between;  
  align-items: center;  
}
```

Extended — all 4 sections

```
.site-header { display: flex; align-items: center; gap: 16px; background: #1a1f29; padding: 14px 20px; }  
.site-title { font-weight: 700; }  
  
.site-nav { display: flex; gap: 16px; }  
  
.account-card { background: #1a1f29; border-radius: 12px; padding: 24px; display: flex; justify-content: space-between; align-items: center; }  
.account-balance { font-size: 32px; font-weight: 700; }  
  
.txn-list { list-style-type: none; }  
.txn-list li { display: flex; justify-content: space-between; }  
.txn-amount.positive { color: #1a8a4c; }
```

Bonus — The Stylesheet Bug

The CSS already has display: flex, justify-content, and align-items — but one property is misspelled and the browser silently ignores it, which is why the card looks stacked instead of laid out side by side.

```
- display: flex;
+ display: flex;
```

THE HELPDESK

What it is: a completely different mechanic from the other four — a live banking app is broken in five ways, and the participant plays with it, describes what's wrong to a support chat in their own words, and the chat summarises the problem, applies the fix directly to the code, and shows a diff of exactly what changed. No manual find-and-edit required. Two more "issues" are actually just polish suggestions, not bugs, to reward people who suggest improvements as well as spot faults.

There's no visible bug list. The old version showed a checklist naming all 5 known issues, which made the activity pointless — you could just read the list. Now there's only a progress bar showing *how many* have been found, never *what they are*. Discovery has to come from actually playing with the app.

THE 5 REAL BUGS — NOW RANDOMISED, TWO VARIANTS EACH

Each session, one of two genuinely different mistakes is picked at random per bug — not just different wording, different actual code. A participant who compares notes with someone else may see a different broken line for the same symptom. Both variants below are real; recognise either.

A real, disruptive bug was found and fixed in the transfer validation fix specifically: the fix originally rejected bad amounts with a real, blocking browser `alert('Enter a valid amount')`. Helpdesk's own "Check My Fixes" verification actually simulates entering a negative and an oversized transfer to test that validation works — so once that fix was applied, running the verification would genuinely pop up that alert, interrupting the person mid-check with no warning. The `alert()` has been removed entirely from both variants of the validation fix; it now rejects silently, and the underlying logic is unchanged and still correctly blocks bad transfers.

Deposit doesn't add up correctly

Symptom: depositing money doesn't correctly increase the balance.

Variant A — overwrites instead of adding:

```
- balance = amt;
+ balance += amt;
```

Variant B — silently adds zero instead of the real amount:

```
- balance += 0;
+ balance += amt;
```

Withdraw does nothing at all

Symptom: clicking Withdraw has no effect.

Variant A — the button calls `handleWithdraw()`, but the function is defined as `handelWithdraw()`, a typo:

```
- function handelWithdraw(){
+ function handleWithdraw(){
```

Variant B — the function name is correct, but it reads the amount from the wrong input box:

```
- const amt = Number(document.getElementById('depositAmt').value);
+ const amt = Number(document.getElementById('withdrawAmt').value);
```

Most recent transaction missing from the list

Symptom: one transaction is always missing from the list.

Variant A — the loop stops one entry early, so the newest is missing:

```
- for (let i = 0; i < transactions.length - 1; i++) {
+ for (let i = 0; i < transactions.length; i++) {
```

Variant B — the loop starts at position 1 instead of 0, so the oldest is missing:

```
- for (let i = 1; i < transactions.length; i++) {
+ for (let i = 0; i < transactions.length; i++) {
```

Transfers can push the balance negative

Symptom: negative or oversized transfers go through with no error.

Variant A — no validation exists at all:

```
- const amt = Number(document.getElementById('transferAmt').value);
- balance -= amt;
+ const amt = Number(document.getElementById('transferAmt').value);
+ if (amt > 0 && amt <= balance) {
+   balance -= amt;
+ } else {
+   alert('Enter a valid amount');
+   return;
+ }
```

Variant B — a check exists but is ineffective, only blocking exactly zero:

```
- const amt = Number(document.getElementById('transferAmt').value);
- if (amt !== 0) {
-   balance -= amt;
- }
+ const amt = Number(document.getElementById('transferAmt').value);
+ if (amt > 0 && amt <= balance) {
+   balance -= amt;
+ } else {
+   alert('Enter a valid amount');
+   return;
+ }
```

Balance displays a long, broken-looking number

Symptom: the balance sometimes shows something like `30.299999999999997` — genuine JavaScript floating-point behaviour, seeded deliberately into the starting balance.

Variant A — no rounding at all:

```
- document.getElementById('balance').textContent = balance;
+ document.getElementById('balance').textContent = balance.toFixed(2);
```

Variant B — rounded to 10 decimal places instead of 2 (still messy, just differently):

```
- document.getElementById('balance').textContent = balance.toFixed(10);
+ document.getElementById('balance').textContent = balance.toFixed(2);
```

THE 2 "IMPROVEMENTS" (NOT BUGS — REWARD NOTICING THESE TOO, NOT RANDOMISED)

Deposit box doesn't clear after use

Not broken, just rough — the amount stays in the input after a successful deposit.

```
transactions.push('Deposited £' + amt);
+ document.getElementById('depositAmt').value = '';
```

Large balances are hard to read

Adds thousands separators. This one depends on the rounding bug being fixed first — the chat will say so if asked about it too early.

```
- document.getElementById('balance').textContent = balance.toFixed(2);
+ document.getElementById('balance').textContent = balance.toLocaleString(undefined,
{minimumFractionDigits:2, maximumFractionDigits:2});
```

HELP & HINTS — WHAT WORKS AND WHAT DOESN'T

This section has been rebuilt twice since it was first written — worth reading in full even if you've seen an earlier version. Construct, Vault, Ledger, and Front now share one menu-driven system:  **Hint**,  **Help**, and  **Write it yourself**, all inside the same chat panel.

HOW IT WORKS NOW

 **Hint** posts the same tiered, gated hint sequence as before — clicking repeatedly (with a genuine Run between tiers) walks through hint 1, 2, 3.

 **Help** opens two categories, **Syntax** (how do I write this) and **Troubleshooting** (why isn't this working), each listing that module's real reference topics — clicking one shows the answer directly, no typing needed.

 **Write it yourself** is a genuine free-text box — this was removed at one point in favour of pure menus, then restored after feedback that the pure-menu version answered far too little. Typing "I'm stuck" here still drives the exact same hint progression as the button.

What it still can't do: understand a genuinely novel phrasing that doesn't overlap with any known keyword. That gets an honest fallback pointing back at the Help menu — not silence, but not magic either.

A REAL, MEASURED COMPREHENSION BUG — FOUND AND FIXED

Free text was matching far less than it should have, and there was a provable reason: a stress test of 20 realistic phrasings failed 30% of the time before investigation. Three separate causes, all fixed: (1) the normaliser was silently deleting `>=`, `<`, and other symbols before matching — asking "what does `>=` mean" became "what does mean", losing the entire question; (2) the word "if" was both hard-coded into the stopword list and filtered out again by a length rule, so no if/else question could ever match; (3) plurals didn't match singulars at all — "explain booleans" failed against a keyword list that only had "boolean". All three fixed (symbols are translated to words first, "if" is no longer blocked, and a lightweight stemmer now treats "booleans"/"boolean" as the same word) and re-verified with the same battery plus a second, broader one — 100% on both afterward, with off-topic nonsense still correctly falling back rather than false-matching.

VAULT AND LEDGER'S HARDEST EXTENSION CHALLENGES — THE ACTUAL CAUSE, NOT A VAGUE "MAKE IT EASIER"

Both modules quietly introduced a brand new syntax element for the first time in an extension challenge, cold, with no warmup: Vault's `applyLoyaltyDiscount` suddenly needed a ternary; `flagSuspiciousActivity` suddenly needed `&&`; Ledger's `sumByCategories` suddenly needed `.includes()`. None of these had appeared anywhere in the taster. For Vault, the tests never actually required the new syntax at all — they only check output values — so the hints now explicitly say "you already know how to do this with if/else, that works fine" before presenting the shortcut as optional, not required. Both approaches were verified through the real test-running code, not just reasoned about. Ledger's `.includes()` had no such familiar fallback, so instead the brief now introduces it in total isolation, with a standalone true/false example, before the real task ever appears.

Follow-up after continued difficulty reports on these same challenges: all 6 of the hardest extension challenges (Vault's last 3, Ledger's last 3) now have a 4th hint tier, inserted just before the final full-answer hint — a completely worked example using real plugged-in numbers (e.g. `flagSuspiciousActivity` walks through `amount=6000`, `country='US'` step by step, arriving at true, then re-checks with `country='UK'` to show why it flips to false). This sits between "here's the concept" and "here's the code," rather than jumping straight from one to the other. All four tiers were confirmed to deliver correctly in sequence through the real hint-gating mechanism, not just read for accuracy.

Second follow-up, same 6 challenges: now at 6 hint tiers total, not 4. A 5th ("fill in the blank," e.g. `return isPremium ? amount * ___ : amount;`) and a 6th ("a common slip on this one is...", naming a specific realistic mistake like writing `&` instead of `&&`) were both added after repeated requests for more. All 6 tiers per challenge confirmed present in the shipped files and delivering correctly in sequence, not just assumed from having built them once before.

"Is the take-home actually different from the taster?" is no longer something to just explain — it's now visible on screen. All 4 take-home files (Vault, Ledger, Construct, Front Extended) show a clear amber  New — not in the taster version" marker at the exact point new content begins. For Vault/Ledger/Construct that's per-challenge (challenges/levels 4 onward); for Front Extended, which is a single continuous page rather than one-at-a-time challenges, it's a banner near the top naming exactly which sections are new (1, 2, and 4) and which one substantially overlaps the taster (3, the Account Card). Confirmed the badge shows and hides at the correct boundary in each file, and correctly stays hidden during Construct's bonus round, which is a separate mechanic from this taster-vs-take-home comparison.

HELPDESK CHAT — WHAT WORKS AND WHAT DOESN'T

This is now free-text only — there is no menu at all. An earlier version added a clickable button menu (pick an area, then pick the symptom you'd observed) to make blind clicking less trivial, but feedback was that a 10-year-old still blitzed through it just by clicking. The whole menu was removed entirely, not just hidden — opening the chat shows a text box and nothing else. Describing a problem gets: a one-line summary, the fix applied directly into the

code editor, and a red/green diff.

HOW IT BEHAVES

Recognised problems and improvements (7 total, see the answer key above): each has roughly 7-12 different phrasings it recognises, across whichever of the two random variants is actually active that session.

"I'm stuck" is the only way to reach the progress-aware nudge now — there's no dedicated button for it anymore, so the phrase list was deliberately widened ("what am I missing", "what's left", "not sure what else") since this is the sole door into that feature.

Fixes stack correctly: describing a second problem after the first applies its fix on top, without undoing it.

General debugging questions are now answered too, not just bug descriptions: "what is a bug", "what does the diff mean", "what does Check My Fixes do", "what is a function/variable" all get direct answers — added after feedback that this chat should "help like the others" do with general concepts. Checked in this order: stuck-phrase → bug/improvement match → general concept → honest fallback, so a real bug description is never mistaken for a concept question.

What it can't do — genuinely open-ended interpretation: still keyword matching underneath, just with a wide net. It cannot diagnose a problem it wasn't built to recognise.

Manual editing still works alongside chat: participants can type directly into the code window instead of using chat at all — "Check My Fixes" runs the same checks either way.

Badges now exist here too: finding and fixing all 5 real bugs shows a gold badge and an explicit Return Home button, matching every other module — this was missing for a while and has been added.

EXAMPLE EXCHANGES

"clicking withdraw does nothing at all" → diagnoses the handelWithdraw typo, fixes it, shows diff

"large numbers are hard to read" → works only after the rounding bug is already fixed; otherwise a graceful "can't find that" message

"what is the weather today" → honest fallback, no fabricated fix

A SEPARATE "I'M STUCK" PATH, FOR THE HARDER-TO-SPOT BUGS

Feedback during testing was specific: for someone with no coding background, finding the last one or two bugs by clicking around is genuinely hard — the transfer-validation and missing-transaction bugs especially don't announce themselves the way a visibly broken button does. Rather than add them to a visible list (which would recreate the exact spoiler problem the redesign removed), there's now a second, separate detection path in the same chat.

How it works: phrases like "I'm stuck", "help", "what should I look for", or "I can't find any more bugs" are checked *before* the normal fix-matching — if recognised, the chat looks at which of the 5 real bugs haven't been found yet and nudges toward exploring one of them, without naming it. "Try transferring something unusual — a negative amount, or more than the balance actually has. What should happen, and what does?"

It tracks real progress, not a script: the nudge is generated from the same pass/fail state "Check My Fixes" already tracks — it will never point someone back at a bug they've already fixed, and once all 5 are genuinely found it says so plainly instead of repeating itself.

"im stuck" (nothing fixed yet) → nudges toward the deposit bug specifically

"can't find any more bugs" (3 of 5 fixed) → nudges toward one of the 2 remaining, never a fixed one

"help" (all 5 fixed) → "you've actually found everything I know about"

THE TWO COPILOT MODULES

These are genuinely different from the other seven — no code, no tests, no hint system. The point is direct, hands-on proof that prompt quality changes output quality, using the same spreadsheet or inbox every time regardless of what's typed.

EFFECTIVE COPILOT PROMPTS (REFERENCE GUIDE)

A static guide, no interactivity beyond copy buttons — five techniques (specificity, context, format, breaking into steps, iterating), each with a genuine bad-vs-good prompt pair and a "why it works" note. Nothing to troubleshoot here; if a participant asks a question about it, the content is self-contained enough to read together on screen.

EFFECTIVE COPILOT — EXCEL & OUTLOOK (LIVE EXERCISE)

Redesigned since the original version — no typing anywhere now. The original free-text version (where a participant typed their own prompt) was replaced after feedback that typing shouldn't be an option here. Every interaction is now a fixed, large, colour-coded choice card — 😊 Bad / 😐 Average / 😄 Great — showing the exact prompt text before you click it. There is still no real AI call happening: each button is wired to a specific, pre-verified response. If a participant asks "is this really CoPilot," the honest answer is no — it's a simulation built to demonstrate the *relationship* between prompt quality and output quality, not a live model.

Excel exercise — three fixed tiers

TIER	PROMPT SHOWN	RESPONSE
😊 Bad	"Do something with this."	A confused, "what would you like me to do?" response — not just weak, genuinely stuck
😐 Average	"Summarise this data."	A generic text summary, no chart
😄 Great	"Build a dashboard with a chart of sales by region and the quarterly trend, and call out the best performer."	A full dashboard: two real SVG charts plus three stat callouts

The numbers are genuinely computed, not decorative: West is the real top region at £246,000 total; Q4 is the real strongest quarter at £209,000; the grand total is £731,000.

The download link genuinely didn't work, and the first fix attempt was wrong. First symptom reported: downloaded as `RegionalSales.html`. That was misdiagnosed as a missing filename on the `download` attribute; fixing that didn't help — the real symptom afterward was "couldn't download, no file," and the file never appeared as its own item anywhere accessible. **Actual cause:** the link used a normal relative path (`href="RegionalSales.xlsx"`), which only resolves when the whole output folder is downloaded and opened together as real sibling files — it has nothing to point to when the HTML page is being viewed on its own, e.g. in an in-chat preview, since there's no accompanying file present in that context. **Actual fix:** the spreadsheet's real binary data is now embedded directly in the page as a base64 `data:` URI, so the download works regardless of whether the file is sitting alongside it or not — genuinely self-contained, matching how every other flat-file page in this suite already works. Verified by extracting the embedded

data back out of the shipped HTML and confirming it decodes to a byte-identical, fully valid spreadsheet with the correct figures.

Worth remembering for any future file this suite ever needs to offer as a download: a plain relative `href` to a sibling file is not reliable across every way these pages might be viewed. Embedding the data directly (as done here) is the dependable approach.

Outlook exercise — two separate three-tier groups

Find and Summarise are now two visually distinct groups of three cards each, not one combined box guessing at intent from wording.

GROUP	TIER	PROMPT	RESPONSE
Find	☹ Bad	"Find something."	Confused — nothing to search for
	😊 Average	"Find the marketing email."	3 possible matches shown — genuinely ambiguous, there are 3 marketing emails
	😊 Great	"Find the email that needs a signature before Friday."	Uniquely finds the Legal Team contract email
Summarise	☹ Bad	"Tell me what's going on."	Confused — no conversation named
	😊 Average	"Summarise my emails."	"Too broad" response naming the separate conversations that exist, but summarising none of them
	😊 Great	"Summarise the marketing campaign feedback thread."	Correctly merges all 3 thread messages into one summary

The confidence bug from the free-text version is fully retired along with that version — since every response is now a fixed, individually-verified mapping rather than live scoring, the class of bug where an ambiguous prompt could be shown as a false single confident match can no longer occur here. Worth knowing the history if this comes up, but nothing to check for in this version.

Two new closing sections, added after feedback

"Prompts worth keeping" — four copyable, reusable prompt templates a participant can take into real Excel or Outlook, not just this exercise.

A real downloadable file — RegionalSales.xlsx — the exact same figures as the on-screen exercise, built as a genuine spreadsheet with live `SUM` formulas (not hardcoded totals), so it recalculates if someone edits it. Verified with LibreOffice's recalculation check (zero formula errors) and the computed totals independently confirmed to match the browser exercise exactly. Meant for participants to open in real Excel and try prompting on their own data afterward.

"Finding old chats and files" section — five example prompts for searching Teams and Outlook history specifically (a person + topic + rough date; a half-remembered file name; searching both apps at once; summarising a dormant channel; checking your own history before re-asking a colleague). This was requested content, not an interactive exercise — read it together if it comes up, no verification mechanics involved.

THE INTRO SLIDE DECK

`intro-slides.html`, linked from the dashboard header — a 7-slide presentation for opening a session live, distinct from `start-here.html` (which participants read at their own pace). Dark theme, arrow-key and click navigation, a

progress bar, and clickable dot navigation to jump to any slide directly.

The financial-stations slide originally never mentioned the CoPilot modules at all — someone glancing at just that slide would reasonably think there were only 5 things on offer. Fixed with an explicit "⚙️ Plus 2 more on AI skills — next" teaser at the bottom of that slide, and the CoPilot slide itself was expanded from a single paragraph into two proper cards naming both modules directly.

Navigation logic is unchanged and re-verified across both redesigns, not just visually restyled: the actual slide-advancing code was deliberately left untouched both times it was already tested — and re-run through the same boundary tests after each visual rebuild (can't overshoot the last slide, can't undershoot the first, dots and progress bar stay in sync) to confirm neither rebuild broke it.

The first visual redesign wasn't bold enough — feedback was that it still read as "plain black slides."

Rebuilt a second time with a much more deliberate visual direction: a retro-terminal aesthetic leaning into the "Mainframe" name itself — scanline overlay, a subtle animated grid background, neon glow text (cyan/magenta/green), a blinking cursor on the title slide, and corner HUD brackets. Genuinely more dramatic than the first pass, not just an incremental tweak.

CERTIFICATES & MY TROPHIES

What it is: every taster and take-home completion screen (all 9 activity files) now has a genuine certificate — a name field and a "🏆 Claim Certificate" button — alongside the existing badge. Claiming writes an entry (module, tier, name, date) to `localStorage` under the key `mainframe_trophies` . `my-trophies.html` , linked from the dashboard header, reads and displays everything claimed so far.

Trophies deliberately reset when the My Trophies page itself is refreshed. The page reads whatever's in storage, displays it, then immediately clears the underlying data. Viewing it once shows what's been earned; reloading it shows nothing. This is intentional, not a bug — it means the next person using the same shared computer gets a clean slate without needing a separate "reset" step. The page carries an on-screen warning about this; worth mentioning out loud if you're demonstrating it live, since it's easy to lose your own trophies by refreshing to show someone else.

Two real bugs were found and fixed while building this, not just the feature added: Construct's certificate screen (both taster and full build) had a leftover dead reference to a `cadet` name variable from an old, removed flow — harmless only by accident, and would have thrown a genuine error the moment the real certificate input replaced the old placeholder it was pointing at. And Front's "Try Again" button didn't reset the claim button's state, meaning a second person redoing the activity on the same shared computer couldn't claim their own certificate — the button would still show the first person's "already saved" state. Both fixed and tested.

The dashboard link to My Trophies was itself missing for a period — the page existed and worked, but nothing on the dashboard pointed to it. Fixed; it now sits in the header row alongside Session Intro and Facilitator Notes. Worth checking this link is still present if the dashboard header is ever edited again, since it's exactly the kind of thing that's easy to silently lose.

DASHBOARD STRUCTURE

What it is: `index.html` is organised into four labelled sections, in this order: **Start here** (Hello, Computer alone), **Core activities** (Construct, Front, Helpdesk), **AI skills** (both CoPilot modules), and **Advanced extensions** (Vault, Ledger). This replaced an earlier flat grid that listed everything as equally-weighted, in a different order, with no distinction between a 3-minute warm-up and a more involved take-home challenge.

Vault and Ledger are deliberately framed as "a good next step, not a starting point," not as lesser activities. Their difficulty was NOT reduced — that was raised and explicitly declined. The framing is about sequencing (try these once you're comfortable with the core activities), not about watering down the content.

start-here.html had drifted badly out of sync with this structure and needed a genuine rebuild, not a word-level fix: it still described "five short activities" with no mention of Hello, Computer at all, and listed the coding activities in the old flat order with no core/advanced distinction. Rebuilt to match the dashboard exactly, including matching numbering (00 through 07) in the same section order. If the dashboard's section order or numbering is ever changed again, check start-here.html at the same time — this is exactly the kind of drift that's easy to introduce silently.

TROUBLESHOOTING — USER ERROR, NOT THE BUILD

Every one of these has come up in testing and is a person doing something reasonable, not a bug in the files.

"I clicked Run and nothing happened."

Check the Error box beneath the buttons — a syntax error (missing bracket, missing semicolon) stops the code from running at all, and the message is usually specific enough to spot the line. If the Error box is empty too, check they actually edited the function body — an empty `// TODO` with nothing added runs fine and just does nothing.

"My answer looks right but it's still failing."

Almost always exact wording/case: `"Groceries"` won't match `"groceries"`, and a typo'd function name means the test can't find the function at all — the error message will say so directly.

"The page looks completely unstyled / broken, no colours, no layout."

These are self-contained single files with no other dependencies except a Google Fonts import — this is almost always a font failing to load over a blocked network. Check internet access on that machine.

"I can't type in the code box."

Click directly inside the text area first — cursor focus can land on a button instead, especially on a trackpad. Not a bug, just needs a click.

"I hit Reset and lost my code."

Working as intended — Reset wipes back to the starter code for that task, not just the visuals. Worth warning the room about this once up front rather than after someone loses a few minutes of work.

"The test console shows a weird value like NaN or undefined."

`NaN` usually means a maths operation happened on something that wasn't a number (often a typo'd property name). `undefined` almost always means a missing `return`.

"Nothing in The Front changes when I edit the CSS."

Check the selector actually matches — `.account-card` needs the dot; without it, it silently targets nothing.

"I copied the worked example exactly and it still failed."

Working as intended, not a bug — Front's worked examples deliberately use values too small (or, for Layout, a different

value entirely) to pass the real checks. Explain this is on purpose: the pattern is right, the numbers need adjusting.

Construct taster vs. full build — genuinely different levels 4 and 5, not just "full has one extra": both share Wake Up, The Path, and The Turn, but then diverge — the taster's own levels 4–5 are The Relay and The Repeat (documented above), while the full build has Agents and its own version of The Loop instead. The full build's Loop got the identical staircase redesign as the taster's Repeat, for the same reason (a straight line didn't feel meaningfully harder) — just slightly longer (5 diagonal cycles instead of 4), since it's untimed take-home work. Loop's answer is the same shape as Repeat's: a 4-line loop body of move/turn/move/turn-back, repeated, or the fully manual 20-line equivalent — both genuinely pass. The taster is a genuinely different file ([mission-control-taster.html](#)) with its own 6-minute timer. Don't hand out the full [mission-control.html](#) link during the live event by mistake — its timer now counts up with no deadline, which will look odd if someone's expecting a countdown.

ALL FILES

[start-here.html](#)

— session intro, show this first

[index.html](#)

— the dashboard

[hello-world.html](#)

— mandatory first activity, explains what a console is

[mission-control-taster.html](#)

— Construct, live taster (5 tasks + bonus)

[mission-control.html](#)

— Construct, full/take-home (6 levels: shares 1–3 with the taster, diverges at Agents/The Loop, plus new 6th level The Rectangle)

[the-vault.html](#)

— Vault, live taster (5 challenges)

[the-vault-extended.html](#)

— Vault, take-home (6 challenges)

[the-ledger.html](#)

— Ledger, live taster

[the-ledger-extended.html](#)

— Ledger, take-home

[the-front.html](#)

— Front, live taster (colour picker, extensions)

[the-front-extended.html](#)

— Front, take-home

[the-helpdesk.html](#)

— Helpdesk, chat-driven auto-fix, randomised bugs

[copilot-prompts-guide.html](#)

— CoPilot prompting reference guide, no code

[copilot-excel-outlook.html](#)

— CoPilot live exercise, Excel dashboard + Outlook find/summarise

[RegionalSales.xlsx](#)

— real downloadable spreadsheet used by the Excel/Outlook exercise, embedded as base64 inside that page

[my-trophies.html](#)

— trophy case, linked from the dashboard header, resets on its own refresh

[intro-slides.html](#)

— interactive intro slide deck for presenting the session live, linked from the dashboard header

[construct-guide.html](#)

— participant reference

[vault-guide.html](#)

— participant reference

[ledger-guide.html](#)

— participant reference

[front-guide.html](#)

— participant reference

[construct-help.json](#)

— chat knowledge base + synonym table, edit then re-embed

[vault-help.json](#)

- chat knowledge base + synonym table, edit then re-embed
ledger-help.json
- chat knowledge base + synonym table, edit then re-embed
front-help.json
- chat knowledge base + synonym table, edit then re-embed

All files are flat, self-contained HTML — no server or build step. Copy the whole folder anywhere and it works.